Blog Application

Complete Solution Guid

Python | Django 3.x | SQLite | Pillow | Jenkins CI

This document contains the complete, ready-to-use solution for the Django Blog Application capstone project. Copy each file into the corresponding location in **/home/labuser/Desktop/Project/fullstackdev-capstone/**. The project Django package is named **Play** and the app is named **blog**.

Project Structure

Path	Description
`Play/urls.py`	Project-level URL config
`Play/settings.py`	Django settings (do NOT modify)
`blog/models.py`	User & Blog database models
`blog/forms.py`	loginForm, signUpForm, createBlog
`blog/views.py`	All 5 view functions
`blog/templates/signup.html`	Sign-up page
`blog/templates/login.html`	Login page
`blog/templates/index.html`	Blog listing page
`blog/templates/blog.html`	Blog detail page
`blog/templates/create.html`	Create blog page

1. models.py — blog/models.py

Two models are required: a custom **User** model (with email as primary key) and a **Blog** model with a ForeignKey (many-to-one) relationship to User.

```
from django.db import models

class User(models.Model):
firstname = models.TextField()
lastname = models.TextField()
username = models.TextField(unique=True)
email = models.EmailField(primary_key=True)
dob = models.DateField()
my_photo = models.ImageField(upload_to='user_photos/', blank=True, null=True)
# password stored as plain text (no Django auth used)
password = models.TextField(default='')

def __str__(self):
return self.username

class Meta:
db_table = 'user'

class Blog(models.Model):
title = models.TextField()
content = models.TextField()
created_at = models.DateField(auto_now_add=True)
blog_photo = models.ImageField(upload_to='blog_photos/', blank=True, null=True)
blogger = models.ForeignKey(
User, on_delete=models.CASCADE, related_name='blogs')

def __str__(self):
return self.title

class Meta:
db_table = 'blog'
ordering = ['-created_at']
```

2. forms.py — blog/forms.py

Three form classes are required, all inheriting from **forms.Form** (not ModelForm) using the exact class names shown below.

```
from django import forms

class loginForm(forms.Form):
username = forms.CharField(max_length=150)
password = forms.CharField(widget=forms.PasswordInput())

class signUpForm(forms.Form):
firstname = forms.CharField(max_length=100)
lastname = forms.CharField(max_length=100)
username = forms.CharField(max_length=150)
email = forms.EmailField()
password = forms.CharField(widget=forms.PasswordInput())
confirm = forms.CharField(widget=forms.PasswordInput())
dob = forms.DateField(
widget=forms.DateInput(attrs={'type': 'date'}))
my_photo = forms.ImageField(required=False)

def clean(self):
cleaned_data = super().clean()
if cleaned_data.get('password') != cleaned_data.get('confirm'):
raise forms.ValidationError("Passwords do not match.")
return cleaned_data

class createBlog(forms.Form):
title = forms.CharField(max_length=300)
content = forms.CharField(widget=forms.Textarea())
img = forms.ImageField(required=False)
```

3. views.py — blog/views.py

Five endpoints are required. Authentication is handled via Django sessions (not Django's built-in auth) because a custom User model is used.

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from .models import User, Blog
from .forms import loginForm, signUpForm, createBlog

# ■■ helper ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def get_logged_in_user(request):
email = request.session.get('user_email')
if email:
try:
return User.objects.get(email=email)
except User.DoesNotExist:
return None
return None

# ■■ signup — route: signup/ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def signup(request):
if request.method == 'POST':
form = signUpForm(request.POST, request.FILES)
if form.is_valid():
if User.objects.filter(
username=form.cleaned_data['username']).exists():
messages.error(request, 'Username already taken.')
return render(request, 'signup.html', {'form': form})
if User.objects.filter(
email=form.cleaned_data['email']).exists():
messages.error(request, 'Email already registered.')
return render(request, 'signup.html', {'form': form})
user = User(
firstname=form.cleaned_data['firstname'],
lastname =form.cleaned_data['lastname'],
username =form.cleaned_data['username'],
email =form.cleaned_data['email'],
dob =form.cleaned_data['dob'],
password =form.cleaned_data['password'],
)
if form.cleaned_data.get('my_photo'):
user.my_photo = form.cleaned_data['my_photo']
```

```
user.save()
return redirect('login')
else:
form = signUpForm()
return render(request, 'signup.html', {'form': form})

# ■■ login — route: login/ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def login(request):
if request.method == 'POST':
form = loginForm(request.POST)
if form.is_valid():
username = form.cleaned_data['username']
password = form.cleaned_data['password']
try:
user = User.objects.get(username=username)
if user.password == password:
request.session['user_email'] = user.email
request.session['username'] = user.username
return redirect('index')
else:
messages.error(request,
'incorrect username or password')
except User.DoesNotExist:
messages.error(request, 'incorrect username or password')
else:
form = loginForm()
return render(request, 'login.html', {'form': form})

# ■■ logout ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def logout(request):
request.session.flush()
return redirect('login')

# ■■ create — route: create/ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def create(request):
user = get_logged_in_user(request)
if not user:
return redirect('login')
if request.method == 'POST':
form = createBlog(request.POST, request.FILES)
if form.is_valid():
blog = Blog(
title =form.cleaned_data['title'],
```

```
    content=form.cleaned_data['content'],
    blogger=user,
    )
    if form.cleaned_data.get('img'):
    blog.blog_photo = form.cleaned_data['img']
    blog.save()
    return redirect('index')
    else:
    form = createBlog()
    return render(request, 'create.html', {'form': form})

    # ■■ index — route: index/ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    def index(request):
    user = get_logged_in_user(request)
    if not user:
    return redirect('login')
    blogs = Blog.objects.filter(blogger=user)
    return render(request, 'index.html', {'blogs': blogs, 'user': user})

    # ■■ blog detail — route: index/blog/<int:id>/ ■■■■■■■■■■■■■■■■■■
    def blog(request, id):
    user = get_logged_in_user(request)
    if not user:
    return redirect('login')
    blog_post = get_object_or_404(Blog, id=id, blogger=user)
    return render(request, 'blog.html',
    {'blog': blog_post, 'user': user})
```

4. urls.py — Play/urls.py

Replace the entire contents of **Play/urls.py** with the code below. All six URL patterns (including logout) are registered here.

```
from django.contrib import admin
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static
from blog import views

urlpatterns = [
path('admin/', admin.site.urls),
path('signup/', views.signup, name='signup'),
path('login/', views.login, name='login'),
path('logout/', views.logout, name='logout'),
path('index/', views.index, name='index'),
path('index/blog/<int:id>/', views.blog, name='blog'),
path('create/', views.create, name='create'),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Note: Add these two lines at the bottom of Play/settings.py (do not change anything else): MEDIA_URL = '/media/' and MEDIA_ROOT = os.path.join(BASE_DIR, 'media')

5. HTML Templates — blog/templates/

Create the following five HTML files inside **blog/templates/**.

signup.html

```
<!DOCTYPE html>
<html>
<head><title>Sign Up</title></head>
<body>
<h2>Sign Up</h2>
{% if messages %}
{% for msg in messages %}
<p style="color:red;">{{ msg }}</p>
{% endfor %}
{% endif %}
{% if form.non_field_errors %}
<p style="color:red;">{{ form.non_field_errors }}</p>
{% endif %}
<form method="post" enctype="multipart/form-data">
{% csrf_token %}
<p>First Name: {{ form.firstname }}</p>
<p>Last Name: {{ form.lastname }}</p>
<p>Username: {{ form.username }}</p>
<p>Email: {{ form.email }}</p>
<p>Date of Birth: {{ form.dob }}</p>
<p>Password: {{ form.password }}</p>
<p>Confirm Password: {{ form.confirm }}</p>
<p>Photo (optional): {{ form.my_photo }}</p>
<button type="submit">Sign Up</button>
</form>
<a href="/login/">Already have an account? Login</a>
</body>
</html>
```

login.html

```
<!DOCTYPE html>
<html>
<head><title>Login</title></head>
<body>
<h2>Login</h2>
{% if messages %}
{% for msg in messages %}
```

```
    <p style="color:red;">{{ msg }}</p>
    {% endfor %}
    {% endif %}
    <form method="post">
    {% csrf_token %}
    <p>Username: {{ form.username }}</p>
    <p>Password: {{ form.password }}</p>
    <button type="submit">Login</button>
    </form>
    <a href="/signup/">Don't have an account? Sign Up</a>
    </body>
    </html>
```

index.html

```
<!DOCTYPE html>
<html>
<head><title>My Blogs</title></head>
<body>
<h2>My Blogs</h2>
<a href="/create/">+ New Blog</a> | <a href="/logout/">Logout</a>
<hr>
{% if blogs %}
{% for post in blogs %}
<div>
<h3><a href="/index/blog/{{ post.id }}/">{{ post.title }}</a></h3>
<p>{{ post.created_at }}</p>
<p>{{ post.content|truncatewords:30 }}</p>
</div><hr>
{% endfor %}
{% else %}
<p>No blogs yet. <a href="/create/">Write one!</a></p>
{% endif %}
</body>
</html>
```

blog.html

```
<!DOCTYPE html>
<html>
<head><title>{{ blog.title }}</title></head>
<body>
<a href="/index/">Back to My Blogs</a>
<h2>{{ blog.title }}</h2>
<p><small>{{ blog.created_at }} |
{{ blog.blogger.firstname }} {{ blog.blogger.lastname }}</small></p>
<hr>
<p>{{ blog.content }}</p>
</body>
</html>
```

create.html

```
<!DOCTYPE html>
<html>
<head><title>Create Blog</title></head>
<body>
<h2>Write a New Blog</h2>
```

```
<form method="post" enctype="multipart/form-data">
{% csrf_token %}
<p>Title: {{ form.title }}</p>
<p>Content: {{ form.content }}</p>
<p>Image (optional): {{ form.img }}</p>
<button type="submit">Publish</button>
</form>
<a href="/index/">Cancel</a>
</body>
</html>
```

6. Jenkins Configuration

Step	Action
1	Open Chrome → http://localhost:8080
2	Run: cat /var/lib/jenkins/secrets/initialAdminPassword Use the output as the admin password
3	Create a Freestyle project named: Blog
4	Source Code Management → Git Repository URL: file:////home/labuser/Desktop/Project/fullstackdev-capstone
5	Add Build Step → Execute Shell (paste the shell script below)
6	Click Apply → Save → Build Now
7	Console Output should end with: Finished: SUCCESS

Execute Shell Script

```
BUILD_ID=dontkillme

# Install all dependencies
pip install django pillow

# Migrate the application
python manage.py migrate

# Run the server in background on port 8001
python manage.py runserver 0.0.0.0:8001 &

# Wait for server to start
sleep 3

# Run the test suite
python manage.py test blog
```

7. Git Commit

After all files are in place, commit to the local git repository from the project root directory:

```
cd /home/labuser/Desktop/Project/fullstackdev-capstone

git add .

git commit -m "Complete Django blog application"
```

Note: The Jenkins job uses file:////home/labuser/Desktop/Project/fullstackdev-capstone as the repository URL, so the git repo must exist in that exact path.

8. settings.py Additions (Play/settings.py)

Do **NOT** modify anything in settings.py except adding the lines below at the very bottom of the file, and adding **'blog'** to INSTALLED_APPS.

```
# Add 'blog' to INSTALLED_APPS:

INSTALLED_APPS = [

...

'blog', # <-- add this line

]

# Add at the very bottom of settings.py:

import os

MEDIA_URL = '/media/'

MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

9. Final Checklist

#	Task	Done?
1	models.py — User & Blog models created	[ ]
2	forms.py — loginForm, signUpForm, createBlog created	[ ]
3	views.py — signup, login, logout, create, index, blog views	[ ]
4	Play/urls.py — all 6 routes added	[ ]
5	5 HTML templates created in blog/templates/	[ ]
6	settings.py — 'blog' in INSTALLED_APPS, MEDIA_URL/ROOT added	[ ]
7	git add . && git commit -m '...' done	[ ]
8	Jenkins Blog job created and configured	[ ]
9	Jenkins Execute Shell script pasted	[ ]
10	Jenkins Build Now → Console shows Finished: SUCCESS	[ ]